THEME: VIS + AI · INFORMATION VISUALIZATION TERM PROJECT PROPOSAL

LagLens: Seeing What a Robot Actually Executes When Its Policy Thinks Too Slowly

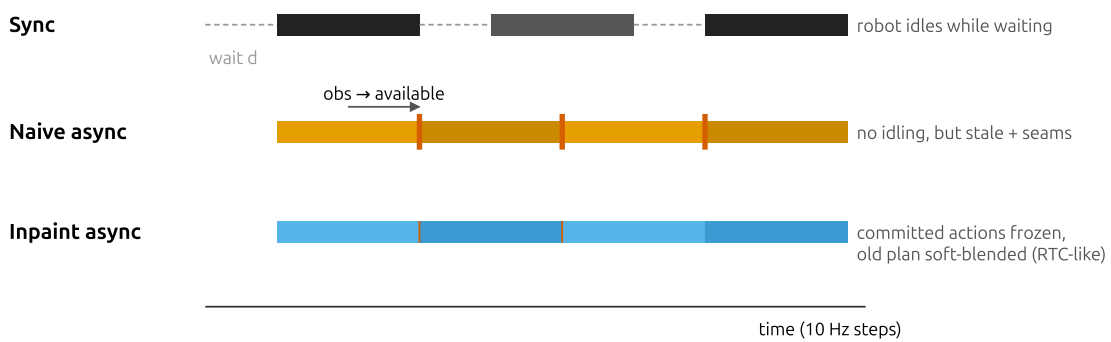
A visual analytics tool for **action-chunk execution under inference latency** (sync vs. asynchronous inference, open-loop execution horizon)

Modern robot policies (Diffusion Policy, ACT, π0, SmolVLA) predict a **chunk** of future actions and execute part of it **open-loop**. Inference takes time *d*, so the robot either **waits** (sync) or keeps executing the old chunk and switches when the new one arrives (**async**).

- **Stale actions**: under async inference an action executed at time τ was planned from the observation at τ−d (VLASH, 2025).
- **Seams**: where chunk *k+1* replaces chunk *k*, the plans can disagree → jumps, mode switches (BID 2025; SEAM 2026).
- **Tuning by hand**: strategy, latency budget and execution horizon are chosen from *aggregate* success-vs-delay curves (RTC 2025; LeRobot async docs).
- **Gap**: today's views are debug plots and tables. We found **no visual analytics work** on chunk execution or async inference (IEEE VIS / EuroVis / CHI search).

Our idea: paired counterfactual replay. We run one pretrained policy from the **same 50 initial states** under **46 execution schedules** and log every predicted chunk. Analysts can then ask: *"Which episodes flip from success to failure when I go async with 400 ms latency, and is it the stale actions or the seams that cause it?"*

Three execution strategies (latency d = 2 steps, horizon h = 4)



What a success-rate table tells you

"naive async, d = 4: 62 % vs. sync d = 0: 72 % success (h = 8)"

What it hides (LagLens)

which of the 50 initial states flipped, when in the episode, and whether stale actions, seams or idling caused it

Self-generated rollout logs of a pretrained diffusion policy

Pretrained `lerobot/diffusion_pusht` run in the PushT simulator on CPU (DDIM 10 steps). Same 50 initial states × (46 schedules + 1 noise replicate) = **2,350 episodes**. Every predicted chunk (16 future 2-D actions) and every executed action is logged.

Attribute (episode table, 2,350 items × 19 attributes)	Type	Order / range / levels	Meaning
strategy KEY	nominal	sync · naive · inpaint	how a new chunk replaces the old one
latency d KEY	ordinal (ratio, steps)	0 < 1 < 2 < 4 < 6 steps (0–600 ms)	inference delay; d = 0 only for sync
horizon h KEY	ordinal (ratio, steps)	2 < 4 < 6 < 8	actions executed open-loop per chunk
replicate KEY	nominal (binary)	0 · 1	1 = sync d0 h8 re-run with new sampling noise (noise floor)
seed KEY	nominal (ID)	1000–1049 (50 levels)	initial state of block & agent (paired)
success	nominal (binary)	0 = fail, 1 = success	coverage > 95 % within 300 steps
coverage_final	quant. ratio, ↑ better	0.00–0.99	overlap of T-block with goal
steps_to_success	quant. ratio, ↓ better	68–300 (missing if failed)	time to finish
seam_gap_mean / max	quant. ratio (px), ↓	2.3–222.2 / 6–305	plan disagreement at chunk hand-over
staleness_mean	quant. ratio (steps), ↓	0.5–9.5	age of obs. behind executed actions
idle_frac	quant. ratio, ↓	0.00–0.75	share of steps with nothing to execute
jerk_mean · path_len · n_inferences	quant. ratio	1.2–221.3 · 205–12828 px · 8–150	smoothness · effort · cost

Dataset type: multidimensional **table** of episodes, each linked to two **temporal** tables and to a **spatial** field (512 × 512 px workspace).

- **Step table** (598,117 items) · keys: schedule, seed, t (ordinal, 10 Hz) · values: agent x, y; block x, y (interval, px); block θ (cyclic, rad); action x, y (interval, px); chunk id (ordinal); index j 0–15 (ordinal); age (ratio, steps); held (binary); coverage (ratio 0–1).
- **Chunk table** (119,373 items) · keys: schedule, seed, chunk id · values: t_obs, t_avail (ordinal time), executed window, 16 predicted (x, y) actions.
- **Size**: 215 KB episode CSV + 112 MB step/chunk JSON (lazy-loaded per schedule).

Keys vs. values: keys = (strategy, latency, horizon, replicate) → *schedule*; × seed → episode; × t → step; × chunk id → chunk. Everything else is a value.

Novel data: no public dataset logs per-inference chunks (survey, Oct 2026). Generated by us; scripts in `scripts/` . 1 step = 100 ms.

Two people who must decide how a policy is executed

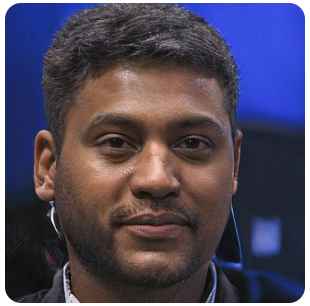


PRIMARY PERSONA

Mara Jensen

28 · 4th-year PhD student, robot learning lab · Copenhagen, Denmark

- **Context**: trains diffusion / flow policies; deploys them on a real arm while the GPU server sits across the building (60–250 ms Wi-Fi latency).
- **Goals**: pick an execution strategy and horizon that keep success high at the latency she actually has; explain *why* in a paper.
- **Needs**: per-episode evidence of what failed, aligned with what the policy planned at that moment.
- **Pain points**: only success-rate tables and video; cannot tell stale actions from seam jumps; re-running real robot trials is costly; matplotlib debug plots show one episode at a time.



SECONDARY PERSONA

Arjun Mehta

37 · ML platform engineer, warehouse-robotics start-up · Seoul (Pangyo), Korea

- **Context**: owns the policy-serving stack (async robot client ↔ GPU server); does not train models.
- **Goals**: set a latency budget (SLO) per robot and a safe default strategy; know when to fall back to sync.
- **Needs**: an overview of how success degrades with latency for each strategy, with confidence that it is not a few lucky seeds.
- **Pain points**: research papers report curves on other robots; no shared vocabulary with researchers about “why it jitters”.

Difference	Mara (primary)	Arjun (secondary)
Expertise	Policy internals (diffusion, chunking)	Systems, networking, latency
Granularity of questions	Single episodes, single chunk hand-overs	Schedules and aggregate trends
Main view used	Workspace replay + chunk timeline (T3, T4)	Schedule matrix + flip matrix (T1, T2)
Decision made	Which strategy / horizon to propose and why	Latency budget and fallback rule

Portraits: StyleGAN-generated, public domain (Wikimedia Commons “Woman 1.jpg”, “Man 2.jpg”); the people do not exist. Personas are fictional.

Four tasks, from schedules down to a single seam

#	Action + target	Concrete question (Mara)	Why	Expected outcome
T1	**Compare → trends** of success / seam gap / staleness across schedules	"How fast does success fall with latency for each strategy, and which horizon is most robust?"	Choose a candidate schedule for the latency the lab actually has.	A shortlist of robust schedules and a latency budget per strategy.
T2	**Locate → outliers**: initial states whose outcome **flips** between a reference and a candidate schedule	"Going from sync d=0 to naive async d=4, *which* seeds start failing, and which are rescued?"	Aggregate rates hide which situations break; paired seeds isolate the schedule's effect.	A small set of fragile seeds to inspect (and to add to a regression test set).
T3	**Identify → features** in time & space: where stale actions, seams and idling occur in a flipped episode	"At which step does the candidate diverge from the reference, and was the executed action stale, a seam jump, or a wait?"	Diagnose the mechanism of failure, not just its rate.	A causal hypothesis, e.g. "a 30 px seam right at block contact makes the push slip."
T4	**Discover → correlation** between seam gaps (plan inconsistency) and task outcome (coverage) over paired episodes	"When a seed moves from R to C, do larger seams go with lower coverage? Does inpainting shrink the seams?"	Recent work claims failed episodes show larger chunk-boundary artifacts (Wang 2026); check it on our policy.	Evidence for or against a strategy (direction and size of shift per seed).

Arjun mainly needs **T1** (latency budget) and **T2** (is the drop driven by a few seeds?). **Mara** chains T1 → T2 → T3 → T4 in one session: overview first, then drill down to a single chunk hand-over (details on demand).

Overview of schedules → paired outcomes → one episode in space & time

View	Task	Idiom & why	Interaction
A · Schedule Matrix	T1	Small-multiple heatmaps, one panel per strategy (spatial region for the nominal key), latency × horizon grid (ordered keys on position). Fill = chosen metric (sequential viridis). 46 cells fit in one glance; a line chart would need 12 overlapping lines.	Click = set candidate C, shift-click = set reference R; radio buttons **change** the metric; tooltip with all aggregates.
B · Paired Flip Matrix	T2	Matrix (seeds × schedules), cells coloured by **outcome relative to R**: gain / loss / both succeed / both fail. Paired design makes flips pre-attentive; rows sorted by robustness. A **noise-replicate** column shows how many flips happen by chance. 50 × 47 = 2,350 cells.	Click column = set C; click row = select seed (→ C, D); sort rows (robustness / seed); hover row = **link** highlight in E.
C · Workspace Replay	T3	Juxtaposed R | C maps of the 512 px workspace at identical scale: goal T, block pose at time t, executed path, and the **chunk fan** (16 planned points; executed solid, discarded dashed). Spatial data needs its own geometry; juxtaposition avoids overplotting two paths.	Scrubber + play (**change** t); hover fan point → tooltip (planned at step, staleness); brushed window from D filters the drawn path.
D · Chunk Provenance Timeline	T3	Gantt-style lanes (R, C) on a shared time axis: thin line = waiting (t_obs → t_avail), bar = executed span, hatch = idle, tick height = seam gap; coverage curves underneath. Shows *which observation every executed action came from*.	**Brush** time → zoom path in C; hover chunk → highlight its fan in C; click → move cursor; tooltip with seam gap.
E · Paired Seed Shifts	T4	Dumbbell plot: one row per seed, ● R value → ● C value of one attribute (default max seam gap, log axis). Rows are grouped by the outcome change R→C (loss / gain / both ✓ / both ✗), so a correlation reads as "do the dumbbells of the failing group point right?", without crossing lines. ▲ marks group medians.	**Brush** rows or click a group → filter B; **change** attribute; click row → open episode; hover → link.

Why this is not a Tableau dashboard: the views are driven by a **reference/candidate pair** of schedules and by chunk **provenance** (each executed action is linked to the observation and the chunk it came from). The fan, the seam ticks and the paired flip colouring are computed from the logged predictions, and selections propagate across all five views.

The most important attribute gets the most effective channel

Attribute (type)	View	Mark	Channel	Justification
latency d, horizon h (ordinal)	A	area (cell)	position (x, y)	ordered keys → ordered spatial position
strategy (nominal, 3)	A	—	spatial region (panel)	top channel for categorical
success rate, seam gap… (ratio)	A	area	colour luminance (viridis)	position already used by keys; uniform, CVD-safe ramp + value label
outcome vs R (nominal, 4)	B	area	hue + luminance	4 levels, blue/vermillion gain/loss; greys for “no change”
seed, schedule (keys)	B	—	position (row, col)	aligned for scanning and sorting
agent / block position (interval)	C	line, point, polygon	position (x, y)	spatial data on true geometry
chunk index j (ordinal)	C	point	luminance ramp along fan	order of planned future steps
executed vs discarded (binary)	C, D	line	line style (solid / dashed)	separable from hue
t_obs, t_avail, exec window (time)	D	line, bar	position on common time axis + length	most accurate for time spans
seam gap (ratio, px)	D	tick (line)	length (height)	magnitude → length
coverage (ratio)	D	line	position (y)	common scale, compare R vs C
seam gap etc. (ratio)	E	point + line	position x (log)	R→C shift as aligned length; log for long tails
outcome change (nominal, 4)	E	row group	spatial region	best categorical channel; no overplotting
success (binary)	C, D	text	✓ / ✗ label	keeps hue free for R vs C
R vs C identity (nominal, 2)	all	all	hue: near-black vs orange	consistent across views, CVD-safe

Colour (Okabe–Ito, colour-blind safe)

Reference R · Candidate C

gain vs R · loss vs R / seam

goal pose · metric ramp

Ranking used (Munzner, Ch. 5):
quantitative: position on common scale > length > angle / area > luminance / saturation
categorical: spatial region > hue > shape

Scalability: A 46 cells · B 2,350 cells · E 50 rows · C/D one episode (≤ 300 steps, ≤ 150 chunks) → all rendered as SVG at interactive rates.

Okabe & Ito (2008) palette · viridis (van der Walt & Smith 2015)

LagLens | R Reference: sync · d0 · h8 ▾ | C Candidate: naive · d4 · h8 ▾ | Matrix colour: (● success ○ seam gap ○ staleness ○ idle) | ? Help | ↺ Reset

A · Schedule Matrix

Sync | Naive async | Inpaint async

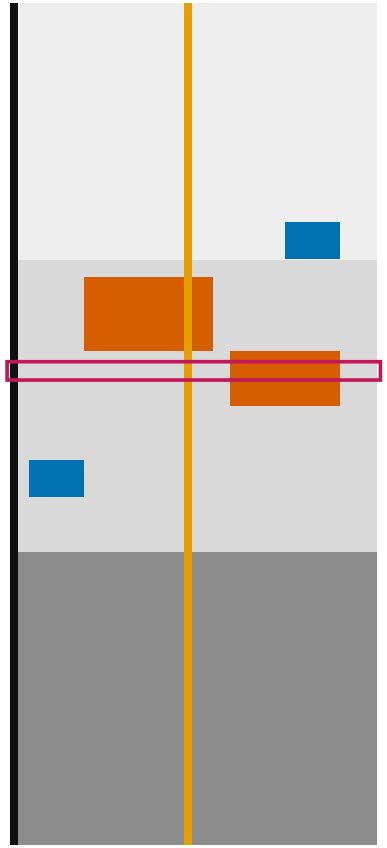


x: latency d (0,1,2,4,6 steps) y: exec. horizon h (2…8)
cell colour: metric chosen in header (viridis)

click cell → set Candidate (orange ring)
shift-click → set Reference (black ring)
hover → tooltip (success, seam gap, staleness, idle)
radio in header → change colour metric

B · Paired Flip Matrix

columns = 46 schedules + noise replicate
rows = 50 initial states, sorted by robustness

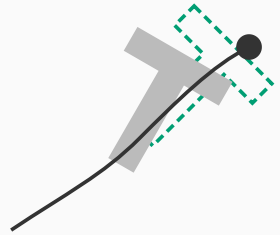


both succeed | both fail
loss vs R | gain vs R

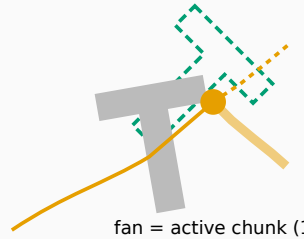
click row → open episode in C, D
click column → set Candidate

C · Workspace Replay (seed 1023) R | C juxtaposed, same scale

R sync · d0 · h8 ✓ success



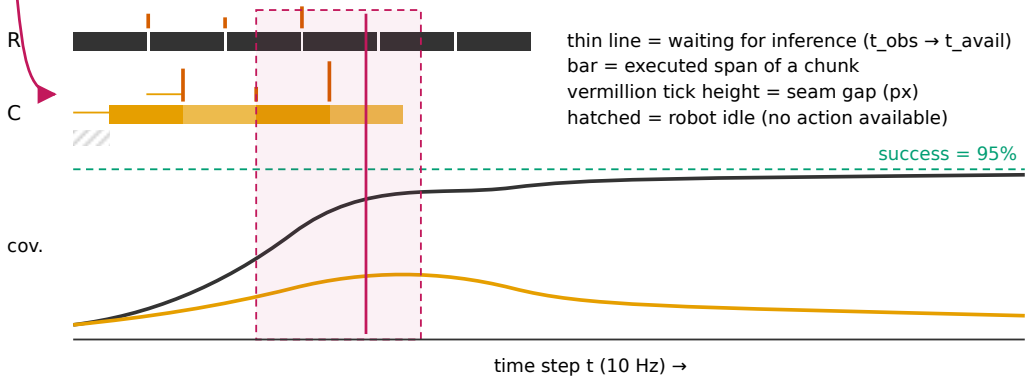
C naive · d4 · h8 ✗ failure



fan = active chunk (16 planned pts)
thick = executed, dashed = discarded

▶ play ⟲ scrub t = 0 … 300 (10 Hz) hover a fan point → planned-at step, staleness

D · Chunk Provenance Timeline

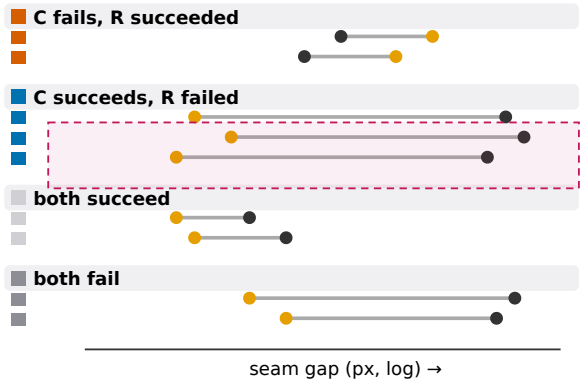


thin line = waiting for inference (t_obs → t_avail)
bar = executed span of a chunk
vermillion tick height = seam gap (px)
hatched = robot idle (no action available)

brush time → show only that window in C · hover chunk → highlight its fan in C · red line = scrub cursor

E · Paired Seed Shifts (dumbbell)

one row per seed: ● value under R → ● value under C
rows grouped by outcome change R→C (same colours as B)



brush rows / click a group → filter B

Flow: A (pick R and C) → B (find flipped seeds) → C + D (inspect where/why) → E (is the shift systematic?). Implemented with d3.js v7 only.